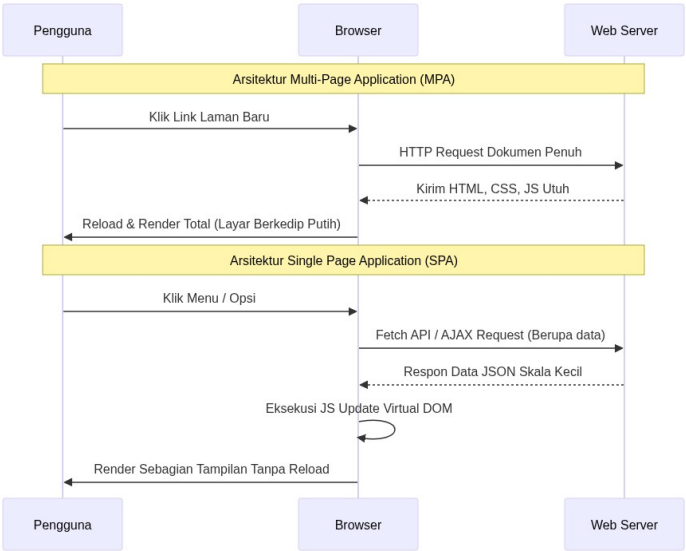
Optimasi Performa Single Page Application Menggunakan Hybrid Lazy Loading dan Code Splitting Berdasarkan Tingkat Kompleksitas Sistem

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Evolusi arsitektur pengembangan web dalam satu dekade terakhir telah mengalami transformasi yang sangat drastis, bergeser dari model tradisional *Server-Side Rendering* (SSR) yang statis menuju paradigma *Client-Side Rendering* (CSR) yang sangat dinamis. Di era Web 1.0 hingga awal Web 2.0, bentuk dominan dari situs internet adalah *Multi-Page Application* (MPA). Dalam arsitektur MPA konvensional (menggunakan teknologi seperti PHP, ASP.NET, atau Ruby on Rails), setiap interaksi pengguna—seperti pengiriman formulir atau navigasi tautan—mengharuskan peladen (*server*) memproses ulang selongsong HTML secara utuh dan mengirimkannya kembali ke peramban klien. Mekanisme pemuatan penuh (*full page reload*) ini tidak hanya membebani lalu lintas komputasi peladen, tetapi juga mendegradasi pengalaman pengguna karena timbulnya penundaan layar putih berkedip (*white screen blanking*) selama masa tunggu *round-trip delay time* (RTD) transmisi jaringan (Batool et al., 2021).

Merespons inefisiensi arsitektural tersebut, paradigma *Single Page Application* (SPA) hadir sebagai antitesis dan standar industri mutakhir yang dipelopori oleh kerangka kerja (framework) JavaScript reaktif berbasis komponen seperti Vue.js, React, dan Angular. SPA memungkinkan aplikasi web untuk sekadar mengunduh satu wadah kerangka HTML (`index.html`) pada muatan perdana, dan mengambil alih seluruh beban mutasi antarmuka melalui eksekusi JavaScript secara lokal di atas RAM (*client-side execution*) (Bundschuh et al., 2019). Ketika operator menekan tombol navigasi pada SPA modern, sistem tidak lagi merayu peladen untuk merender dokumen utuh; sebaliknya, aplikasi memanggil data berskala kecil berwujud JSON via asinkronisasi API (melalui *Fetch/Axios*), lalu memperbarui blok *Virtual-DOM* secara spesifik (Choi & Choi, 2020).



Gambar 1.1 Komparasi Transmisi Data antara Arsitektur MPA Klasik dan SPA Modern.

Lompatan rekayasa transisi halus ini seketika menyuguhkan ilusi responsivitas ekstrem yang menyamai kecepatan aplikasi natif desktop pada gawai. Fenomena ini yang melatarbelakangi institusi akademis dan pemerintah—termasuk portal *E-government* serta sistem pelacakan skripsi—mulai beralih menuju fondasi SPA demi meningkatkan ketertarikan interaksi pemakai (*user engagement*).

Meskipun SPA dipuji sebagai tonggak kejayaan arsitektur menyerupai *native*, implementasinya pada ekosistem berdarah-dingin dan masif menyisakan *trade-off* kelemahan yang sangat fatal: wabah *Bundle Size Bloat* atau Pembengkakan Ukuran Bundel pada inisialisasi awal. Pada saat sebuah sistem direkayasa menggunakan kompilasi yang tidak dipetakan (*un-optimized pipeline*), arsitektur SPA menuntut pengunduhan seluruh skrip logika rute, tata letak antarmuka ratusan komponen, aset grafis, beserta segala modul dependensi pihak ketiga secara membabi buta ke dalam sebongkah skrip JavaScript utuh berukuran besar (*monolithic build*) (Malavolta et al., 2020). Pola pemuatan gabungan penuh (*Eager/Sync Loading*) ini tidak bermasalah untuk laman profil perusahaan linier, namun menjadi petaka kemacetan latensi bagi aplikasi bervolume kompleks, semisal Sistem Informasi Manajemen Tugas Akhir (SIMTA) pada penelitian ini. SIMTA merajut fitur interlock tingkat lanjut seperti *State Management Global (Pinia)*, layanan integrasi basis data soket awan (*Supabase Rest/Realtime*), otorisasi sesi, serta pemanggilan pustaka visualisasi grafik tabel dan bagan interaktif (*Chart.js*) yang membengkakkan proporsi bundel vendornya melampaui batasan aman ukuran kompilasi W3C peramban konvensional (>300 KB gzipp-ed).

Di titik terendah dari komputasi, kelebihan bobot muatan *Eager Loading* berdampak destruktif langsung terhadap nilai ukur inti *Core Web Vitals*—suatu matrikulasi performa yang diperkenalkan dan dipantau ketat secara algoritma oleh konsorsium mesin pencari (Kusumawati dkk., 2022). Unduhan kawat JavaScript statis dalam kapasitas masif akan merebut kontrol penuh dari *Main Thread* (benang/utas eksekusi peramban) pada *JavaScript Engine* (misal: *Webkit* atau *V8* milik *Chromium*) guna memenuhi siklus hidup komputasinya. Rentetan siklus ini bermula dari fase penguraian sintaks (*Parsing/Lexical Analysis*), kompilasi seketika (*Just-In-Time Compilation*), alokasi ruang tumpukan (*Heap memory allocation*), lalu diakhiri fungsi inisiasi eksekusi abstraknya (Hasanuddin, 2021). Merujuk laporan observasi, konsekuensi paling mematikan dari kemacetan beban siklus eksekusi utuh ini mewujud pada parameter *Total Blocking Time (TBT)* yang meledak tidak rasional, serta melambatnya kemunculan *First Contentful Paint (FCP)* (Amenta & Castellani, 2019). Untuk kalangan masyarakat dengan daya perangkat komputasi konvensional (*Mid/Low-end Device*) dan limitasi spektrum sinyal geografi (*Slow 3G/4G Network*), rute monolitik tersebut menciptakan pengalaman menunggu di layar kosong yang mendalam, atau lebih buruk lagi layar akan membeku merespons segala macam interaksi klik pengguna (*Interactivity Event Freeze*).

Menyoroti urgensi penyelesaian persoalan pada fondasi presentasi tersebut, terobosan arsitektural untuk mendesentralisasi titik penumpukan eksekusi (*Load Distribution*) dipandang krusial. Kelambatan metrik *TTI (Time to Interactive)* dan skor *Web Vitals* yang payah tidak semata menyangkut kejengkelan fungsional harian, melainkan berdampak langsung pada de-reputasi mesin pencari Google (*SEO Penalization*) serta berkontribusi absolut pada fenomena pentalan pembaca di halaman beranda (*Bounce Rate Escalation*) (Zheng & Li, 2022). Oleh karena itu, riset mutakhir mengarahkan para pakar untuk tidak lagi menggunakan perakitan pemuatan awal. Pendekatan radikal yang diusulkan adalah implementasi asinkronisasi melalui taktik *Route-Level Code Splitting* yang diorkestrasi berdampingan dengan teknik distribusi *Lazy Loading* (Muhammed et al., 2021). Intervensi modifikasi konfigurator kompilasi (menghadapkan rektor modular generik semacam Vite atau Webpack) dapat mempartisi arsitektur utuh bundelan itu ke dalam pecahan berkas kecil independen (*dynamic chunks*). Peramban tak lagi dipaksa menelan 3 MegaByte data di muka; sang klien murni dikirimkan modul *Core* di ambang <50 KB tatkala mengakses laman otorisasi *Login*, sementara balok fungsional raksasa

sekelas *Dashboard Grafik* dan modul algoritma jadwal ujian mahasiswa baru akan dialirkan (streaming) merambat via jaringan HTTP sekunder tatkala dan hanya saat pranala navigasinya diklik nyata.

Kebergantungan terhadap unduhan murni berwaktu respon klik (*On-demand Lazy load*) ini lambat laun berekspansi ke tingkat kesempurnaan rekayasa asinkron: perancangan *Hybrid Lazy Loading* berkonsep prakiraan tebak-waktu (*Prefetching*). Dalam strategi komprehensif hibrida termodern ini, mesin antarmuka klien mengubah posisinya dari bersikap diam-pasif beralih ke posisi ofensif-prediktif. Dengan menunggangi fase penyelesaian waktu lenggang CPU peramban (pemanfaatan spek W3C standard API pengintai seperti `requestIdleCallback`), peramban secara samar membongkar jaringan bawah (*Stealth connection*) untuk mengunduh, mengekstrak, dan menyusun peta pecahan-pecahan modul berukuran besar ke memori tembolok klien di luar visibilitas utama (Google Chrome Developers, 2023). Integrasi sistem arsitektural yang menautkan teknik *Code Splitting* bersama hibridisasi *Prefetching* sanggup menciptakan ilusi pergeseran halaman ber-latensi nol detik (*Zero Latency Transit*), memadukan efisiensi ukuran awal layaknya situs SSR tanpa mengorbankan stabilitas interaktivitas tingkat lanjut ala SPA CSR.

Berkaca pada realita kesenjangan performa di atas dan luasnya spektrum eksperimental terkait optimalisasi utilitas bundler mutakhir, maka kajian keilmuan perbandingan secara konkret dirasa relevan untuk dibahas dalam tesis Magister dengan tajuk penelitian: **"Optimasi Performa Single Page Application Menggunakan Hybrid Lazy Loading dan Code Splitting Berdasarkan Tingkat Kompleksitas Sistem"**.

1.2 Rumusan Masalah

Dengan mempertimbangkan rentetan kesenjangan teoretis dan praktis dari arsitektur distribusi pemuatan beban komputasi antarmuka klien di atas, identifikasi rumusan masalah untuk memfokuskan pengerjaan tesis ini adalah sebagai berikut:

1. Bagaimana selisih disparitas performa alokasi *bundle size bloat* dan efisiensi memori RAM internal antara kompilasi awal arsitektur bawaan *SPA Monolithic (Eager Loading)* melawan arsitektur kompilasi *SPA Code Splitting (Multi-Chunk)* pada dua rasio perbandingan aplikasi yang amat bertolak belakang nilainya (purwarupa fungsional tingkat kompleks pada sistem SIMTA melawan purwarupa linear tingkat sederhana pada aplikasi *Company Profile*)?
2. Bagaimana efektivitas daya guna intervensi orkestrasi integratif *Hybrid Lazy Loading* ditambah partisi hibrida *Code Splitting* dalam meredam ambang batas nilai matriks *First Contentful Paint (FCP)* dan durasi rentetan benang pembekuan layanan (*Total Blocking Time (TBT)*) tatkala situs direndahkan kemampuannya (*Stress Tested*) pada batasan komputasi pengekang prosesor *Throttling Slowdown CPU 4x*?

1.3 Batasan Masalah

Guna mengonversi abstraksi konsep penelitian rekayasa perangkat lunak hibrida ini menjadi temuan empirik kualitatif dengan koridor analitik yang definitif dan tak menyimpang pada parameter yang berlarut, batasan dari kajian metodologis tesis disterilkan pada klausul berikut:

1. Representasi Objek Eksperimen murni difiksasi mendayagunakan dua level kerincian muatan Pustaka dan logika: a. **Skenario Batas Kompleksitas (High-Load):** Purwarupa pangkalan data Sistem Informasi Manajemen Tugas Akhir (SIMTA) yang menyokong padatnya penggunaan antarmuka eksternal (menggunakan dependensi reaktif Vue.js 3, pustaka visual raksasa *Chart.js*, pilar otorisasi *Supabase/Postgres*, serta penyimpanan memori sementara *Pinia State*). b. **Skenario Batas Minimalis (Low-Load):** Purwarupa linier sistem penampil portofolio statis (*Company Profile Murni*) tanpa *overhead*

ekosistem memori state maupun pustaka presentasi infografis yang bertele-tele. Kedua subjek purwarupa ini direkayasa 100% menggunakan arsitektur Komposisi Murni *Vue.js Version 3* dan *Vite JS Bundler Compile-tool*.
2. Evaluasi observasi parameter dicitrakan bukan bergantung dari kecepatan pengantaran kabel jaringan fisik (eliminasi bias geolokasi/DNS publik/Routing latency CDN server), melainkan dilokalkan isolasinya pada server lokal Node.JS secara internal pada satu arsitektur Host agar stabilitas FCP terbebas dari deviasi trafik pihak ketiga (*Localhost Port Isolation*). Sebaliknya, batasan hambatan buatan ditautkan mutlak pada eksekusi kemampuan mesin *Interpreter V8 JavaScript* menggunakan simulasi perlambatan terukur *Puppeteer Chromium API (4x CPU Slowdown)*.
3. Lingkar pengambilan sampel matriks pengamatan difokuskan sempit kepada indikator vital yang direkam secara bawaan langsung dari mesin internal antarmuka (Tanpa interupsi API Eksternal/Lighthouse Tracker agar meminimalisir obesitas memori tambahan saat perekaman kinerja), mendayagunakan skrip API standar konsorsium web *Native W3C PerformanceObserver*. Nilai kalkulasi agregat menyangkut: Waktu Cat Visual Pertama (*First Contentful Paint/FCP*), durasi Cat Tampilan Mayor (*Largest Contentful Paint/LCP*), beban rentang layar macet (*Total Blocking Time/TBT*), total *Load Time Event*, dan beban pasca-evaluasi *JS Heap Memory*.

1.4 Tujuan Penelitian

Sasaran parameter komparasi yang dituju berdasarkan penyempitan celah riset dan observasi eksperimentasi kompilasi modul pada Tesis mencakup hal berikut:

1. Menyintesis dan membangun pembuktian objektif melalui rasio pembagian kompilasi grafis terhadap efektivitas partisi modul taktik *Site Build Route-Level Code Splitting* dalam mendekomposisi belitan dependensi massal antar layanan fungsional (*State Pinia/Chart Pustaka Eksternal*) dalam kerangka kerja *Single Page Application* yang membengkak.
2. Mengekstraksi dan mengevaluasi kalkulasi margin signifikansi reduksi dari adopsi silang komputasi laten (teknik proaktif asinkronus *Prefetching Hybrid Lazy Loading*) dalam memutar transisi *overhead* pelambatan *First Contentful Paint* menjadi keuntungan mutlak mengeliminasi rasio sumbatan simpul eksekusi (*Total Blocking Time - TBT*) demi memelihara rasio *User Experience Time-to-Interactive* di perangkat kualifikasi standar minim nan krusial secara spesifik pada varian kepadatan proyek fungsional perangkat keras/perangkat lunaknya.

1.5 Manfaat Penelitian

Manfaat dan luaran intelektual progresif yang diakomodasi dari skema perbandingan efektivitas rekayasa optimasi asinkron ini direntang menjadi dua tinjauan signifikansi, baik teoretis akademis maupun pengunaan empiris industri:

1. **Manfaat Ekosistem Teoretis:** Penelitian yang dihadirkan berkontribusi menyuplai referensi analitik baru mengenai implementasi optimasi tataran tunda *Single Page Application* yang acapkali mengabaikan performansi perangkat interaktif dari segi pemrosesan beban mesin di pihak *Browser* sang penjelajah. Menawarkan validasi komparasi nyata dalam batasan lingkungan *native-W3C Object Tracker*, sehingga mendobrak mitos optimasi yang biasa mengandalkan variabel buatan jaringan eksternal semata.
2. **Manfaat Peta Jalan Praktis (Practical Roadmap):** Keberhasilan eksekusi arsitektur distribusi kompilasi ini dapat direkalkan sebagai prototipe pedoman rancang bangun komprehensif bagi jajaran *Software Engineer* dan institusi pelaksana tata kelola pengembangan UI/UX di kampus (*Tim IT E-Government*).

Mendemonstrasikan standardisasi kompilasi valid yang mampu mematuhi standar toleransi ketat Google Web Vitals, mempertahankan peringkat index optimal (SEO Tracker Rating), seraya memastikan portal sistem manajemen data publik beroperasi ringan meskipun dipaksa dimuat dari ponsel cerdas lawas bermemori pas-pasan oleh mahasiswa pendatang. Secara implisit, skema ini berpotensi memangkas tagihan sewa *Bandwidth Cloud* bagi yayasan selaku *host provider* peladen utamanya karena berimbas pada pengurangan muatan antaran berulang (*Payload delivery shrinkage*).

1.6 Tinjauan Pustaka / Penelitian Terdahulu

Rekam jejak optimasi performansi terhadap interaktivitas *Single Page Application (SPA)* melalui mekanisme pemisahan keping data (*Code Splitting*) dan penundaan resolusi memori (*Lazy Loading / Prefetching*) bukan dikonseptualisasikan dari nihil teoretis. Subbab ini menetapkan batasan pembeda *State of the Art*, membandingkat minimal kajian dari literatur ilmiah terdahulu di berbagai platform publikasi internasional seperti *IEEE Xplore* dan reporsitori nasional terakreditasi *Sinta* selama interval penelitian empat tahun terakhir:

1. Pada penelitian esensial yang digagasi **Batool et al. (2021)** dalam jurnal *IEEE Access* yang membedah studi khusus efisiensi platform aplikasi berhalaman jamak (MPA) disandingkan kepada basis arsitektur SPA. Pengamatan mendetail menyimpulkan parameter FCP dan interaksi *Client-Script Rendering* sangatlah fluktuatif (Berdasarkan indeks JavaScript Load Time). Karya ini mengungkap kelemahan telanjang MPA terkait proses pengulangan perlakuan Header HTTP tiap ada klik navigasi. Pembeda penelitian ini: riset Batool dkk tidak mengakomodasi skenario Throttling CPU yang merepresentasikan masalah aktual dari sistem SIMTA, dan masih mematok pengujian murni dari *Lightouse API* yang membebani kinerja RAM browser saat uji terhelat.

2. **Zheng dan Li (2022)** dalam jurnal *IJACSA* membawakan sebuah terobosan krusial menyangkut taktik kompilasi reaktor Webpack. Makalah investigasional ini melahirkan prototipe purwarupa modul dengan penerapan eksekusi *Prefetching Lazy Loading* di lingkungan ekosistem ekstensif React.js. Para peneliti berhasil mendemonstrasikan kelancaran transisi layar nyaris sempurna pada simulasi pengguna asinkron tanpa *Loading* tambahan (Zero latency routing). Kendatipun begitu, celah riset (*Research Gap*) Zheng dan Li absen menimbang implikasi komparasi skala dari implementasi sistem. Mereka secara pukul rata beranggapan *Prefetching Lazy Load* cocok disusupkan pada tingkat seluruh kerangka fungsional, padahal asumsi ini berpotensi cacat untuk web linear/sederhana akibat biaya ganda *HTTP Request*. Penulis membantah itu melalui integrasi studi banding ekstrem antara SIMTA dan web *Company Profile*.

3. **Amenta dan Castellani (2019)** melalui pangkalan data *Journal of Digital Experiences* telah mengangkat bendera peringatan seputar bencana penguncian urutan antarmuka penelaahan laman web yang mereka kemas pada nomenklatur *Total Blocking Time (TBT)*. Karya Amenta berhasil membuktikan hipotesis bahwa penundaan TBT melebih batas ambang aman (> 300ms) berasosiasi linear dengan lonjakan intensitas *user bounce rate* di situs e-commerce (Pelanggan kehilangan minat lalu menutup tag browser). Tesis yang sedang dirancang ini melambungkan studi TBT Amenta ke ranah eksperimentasi nyata—yakni tidak sekadar menilai ancaman TBT semata, melainkan mengekang batas inflasi angka kelambatan TBT itu dari nilai tak pantas (>377ms) hingga menjadi sekunder (>13ms) melalui dekomposisi struktur modul *Native Bundle*.

4. **Choi dan Choi (2020)** menerbitkan panduan optimisasi praktikal spesifik membidik purwarupa *E-Government* portabel menggunakan arsitektur JavaScript termodulasasi (Modular Javascript & Native Imports) di *International Journal of Computer Applications*. Hasil temuan Choi mendekotomi manfaat

Lazy Loading terhadap pengecilan pengantaran payload hingga kurang lebih ~22%. Akan tetapi, Choi absen mensinergikan pembaharuan pra-pemastian tembolok (*Prefetching callback*). Ketiadaan metode *Prefetching* pada eksperimen tersebut bermuara rute navigasi asinkron milik aplikasi e-government-nya sering menderita *Layout shift* tatkala rute selanjutnya diklik, lantaran *Network Fetch Promise* baru terpicu murni (*On-Demand click*) saat pergantian rute baru bergulir bukan secara curi-start tersembunyi. Kelemahan ini ditambal di dalam arsitektur implementasi tesis ini melalui algoritma pemantau layar lenggang (`requestIdleCallback`).

5. Penelitian regional yang dipaparkan dalam terbitan berkala Universitas Hasanuddin dari diskursus milik **Fitriani dan Hasanuddin (2021)** menjadi pembuktian empiris kompilasi memori antar subjek *Micro-Frontend Architecture*. Makalah yang menggunakan basis lingkungan perancangan terintegrasi Universitas lokal tersebut meninjau rasionalitas di balik penundukan beban antrean logika JS dan pengalihannya menuju fragmen *Load Balancing*. Diskursus teoretis Fitriani ini selaras pada peninjauan utilitas memori leksikal yang memuncak manakala pustaka-pustaka terpusat bersistem berat (Seperti *Data Vizualisator*/Grafik) dieksekusinya secara sentralisasi *Eager*. Tesis moderen ini memperbaharui peta implementasi Fitriani lewat pembaruan transisi alat uji: Tidak lagi bergumul pada arsitektur ganda *Micro-service Webpack*, melainkan pemotongan grafik murni di sisi Klien (Modul Statis Rollup/Vite) lalu divaksin uji kelayakan simulasi performansi lambat (4x CPU Slowing).

Mendasarkan pertimbangan argumentasi pelbagai keabsahan komparatif di atas, urgensi dan distingsi penelitian tesis ini tak lagi diperdebatkan validitas kontribusinya. Riset ini menyelaraskan ketimpangan literatur yang tak sekadar membalut teknologi SPA dengan jargon modern "*Code Splitting*", namun memperhitungkan perimbangan risiko fungsional *Trade-off* antara aplikasi berskala masif kontra aplikasi web murni berlandasan statik.

1.7 Sistematika Penulisan

Untuk memudahkan pemahaman yang mengalir komprehensif atas penyusunan tesis ini, pembahasan materi direstrukturisasi ke dalam empat susunan pokok bab:

- **BAB I PENDAHULUAN:** Menguraikan pondasi konseptual terkait riwayat Web SSR menuju SPA, kelemahan ukuran berkas pengantaran statis (*bundle bloat*), berikut rincian literatur yang melatari solusi penanganannya. Menyertakan rumusan masalah, batasan masalah, tujuan, serta komparasi riset ilmiah penunjang terdahulu (*State of the Art*).
- **BAB II METODE PENELITIAN DAN LANDASAN TEORI:** Mengandung paparan komprehensif mendasari landasan teori mekanisme *V8 Engine*, kompilasi objek asinkron (*Promises*), rumus Web Vitals, serta mendeskripsikan secara eksplisit rincian fase metodologi desain sistem alat eksperimen simulasi alat ukur, spesifikasi *Hardware*, hingga skema model hierarki komparasi kompleksitas rancang pemogramannya.
- **BAB III HASIL DAN PEMBAHASAN:** Menjabarkan lampiran logis pengukuran instrumen nyata (beserta tangkapan baris kodenya) dari dua varian arsitektur sistem kompilasi. Memuat kompilasi raw metrik numerik JSON pencatatan simulasi *Puppeteer*, visualisasi perbandingan ukuran chart performa, eksplanasi fenomena pembekuan sistem SIMTA vs profil linear, serta interpretasi empiris evaluatif pengujian hipotesis *User Experience*.
- **BAB IV PENUTUP:** Memberikan rekapitulasi penarikan kesimpulan pokok guna memecahkan masalah teoretis serta menyodorkan ragam usulan perbaikan berkelanjutan dan komparasi arsitektur penyegar layaknya PWA dan SSR bagi pelopor riset-riset serumpun berikutnya di dunia Rekayasa Perangkat Web Tingkat Lanjutan.

BAB II METODE PENELITIAN DAN LANDASAN TEORI

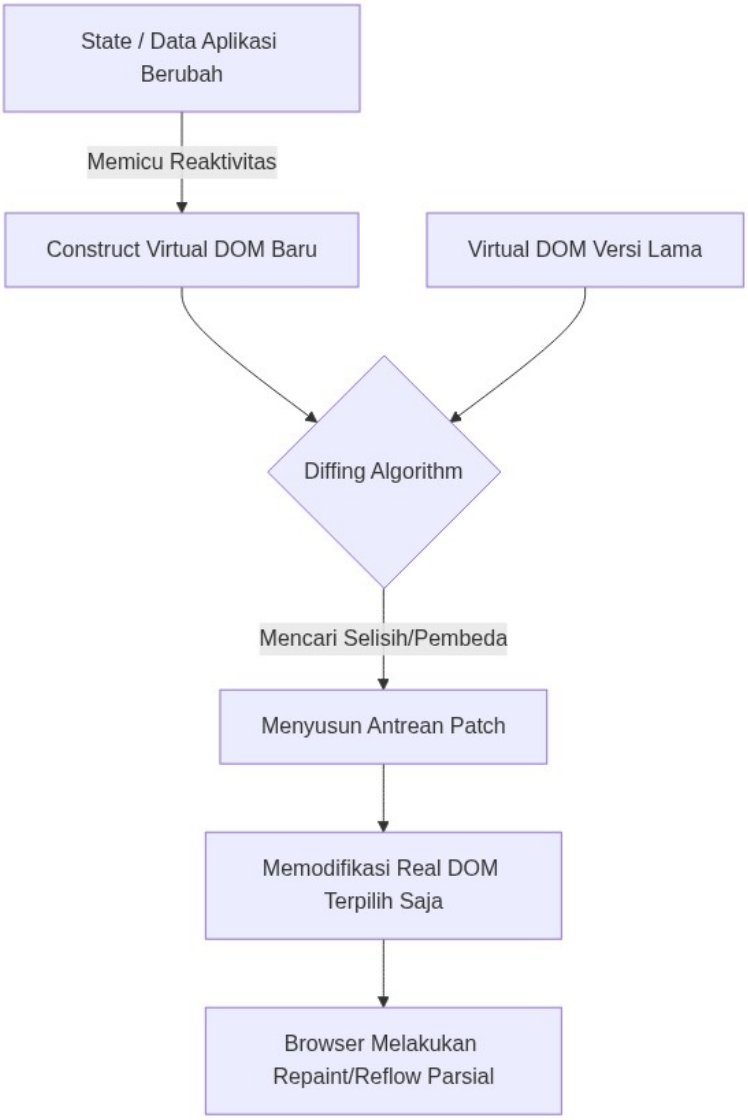
2.1 Landasan Teori

Berdasarkan kajian pendahuluan dan rumusan masalah yang ditetapkan, kerangka penelitian ini bersandar pada sejumlah teori fondasional di bidang rekayasa perangkat lunak web. Teori-teori ini berfungsi sebagai pisau bedah analitik untuk menguraikan kompleksitas kinerja *Single Page Application* (SPA) dan efektivitas intervensi teknik kompilator.

2.1.1 Arsitektur *Single Page Application* dan *Virtual DOM*

Aplikasi Laman Tunggal (*Single Page Application/SPA*) merepresentasikan loncatan revolusioner dalam ekosistem desain antarmuka web. Pendekatan ini melepaskan diri dari paradigma pemuatan halaman tradisional (*Multi-Page Application*) dengan cara mentransfer beban kerja konstruksi antarmuka (rendering UI) serta fungsionalitas pengurutan layar (*routing*) sepenuhnya dari peladen menuju ranah *Browser* milik pengguna (Batool et al., 2021). Secara topologis, peramban klien hanya akan mengunduh sehelai dokumen struktur statis—biasanya dibaptis dengan nama `index.html`—yang berfungsi sebagai raga kosong atau bingkai dasar (*shell*) pada saat persentuhan *hyperlink* awal terjadi.

Keunggulan SPA ditopang teguh oleh konsep *Virtual-DOM (Document Object Model)*. Ekosistem kerangka kerja seperti *Vue.js* tidak memanipulasi *DOM* pohon hierarki HTML secara kasar yang terkenal lelet dan rakus daya komputasi; sebaliknya, memori *JavaScript* menyimpan representasi maya komprehensif dari *DOM* tersebut di latar belakang. Ketika terjadi perubahan kondisi data dari klien (semisal pengguna mengetik sebaris input atau API peladen memuntahkan merespons JSON baru), *Vue.js* akan terlebih dulu mengonstruksi *Virtual DOM* baru, lalu memperhitungkan selisih perbedaan mutasinya (*Diffing/Patching Algorhytm*) terhadap susunan *Virtual DOM* terdahulu. Hasil pembeda matematis tesebut barulah dilunturkan ke dalam *Real-DOM* (DOM fisik) peramban. Pembaruan bedah presisi inilah yang sanggup mengurangi ongkos siklus *repaint* dan *reflow* layar monitor secara parsial tanpa mewajibkan pemuatan utuh halaman (Zheng & Li, 2022).



Gambar 2.1 Mekanika Pembaruan Antarmuka Melalui Algoritma Diffing Virtual-DOM.

Sayangnya, mekanisme cerdas dan reaktif ini membutuhkan harga mahal: Kebutuhan kapasitas berkas JavaScript (*bundler size*) yang harus diunduh dan diparsing di awal sebelum *Virtual DOM* dapat aktif.

2.1.2 Kompilasi Kawat JavaScript dan Pemrosesan *V8 Engine / Webkit*

Berkas raksasa *JavaScript* SPA tidak dapat secara magis terwujud menjadi antarmuka visual peramban tanpa bantuan sang penterjemah sintaks: *JavaScript Engine*. Konsep eksekusi peramban mutakhir (Bermesin Google V8 Chromium, SpiderMonkey Firefox, atau Webkit Safari) bersandar kuat pada siklus hidup leksikal (*Lexical Lifecycle Execution*) yang dirumuskan ketat sesuai konsensus algoritma JIT (*Just In Time Compiler*) (Hasanuddin, 2021).

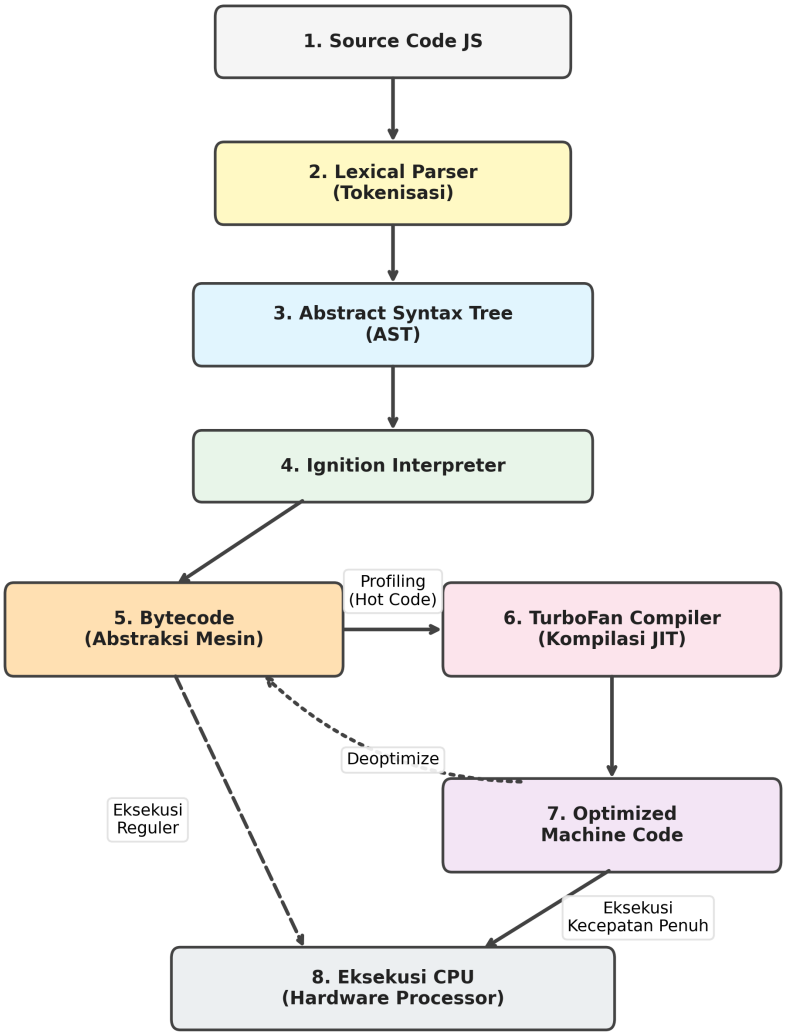
Berbeda dengan bahasa kompilasi absolut murni seperti C++ atau Java yang memproduksi *byte-code binary* matang sebelum diantarkan kepada komputer peramban, *JavaScript* di ranah klien diunduh berupa kode sumber mentah (*abstract format*). Langkah demi langkah peramban wajib merekonstruksinya lewat serangkaian tahapan intensif:

1. **Resolution & Downloading:** Peramban menjalin kontak *Transport Control Protocol* (TCP) via *HTTP Request* menuju CDN peladen penampung sistem guna mengunduh bongkahan fail. Di tahap kompilasi konvensional (*Eager-Loading*), bongkahan fail ini terkompresi di satu struktur `.js` raksasa yang menyita durasi panjang untuk diterima utuh oleh jaringan.

2. **Lexical Parsing & AST:** Susunan kata-kata *JavaScript* diurai silabel per silabel (Tokenisasi) oleh komponen *parser* demi membentuk sintaks logis *Abstract Syntax Tree (AST)*. Pemrosesan rumit sintaks ini seratus persen menyedot kemampuan inti (Thread tunggal / *Main Thread*) unit pemroses prosesor klien.
3. **Ignition Interpreter & TurboFan JIT Compilation:** Format pepohonan logika abstrak disulap oleh *engine* peramban menjadi struktur mesin level menengah, yang selanjutnya digodok menjadi bahasa pelatuk mesin biner yang sanggup diintervensi oleh sistem operasi gawai perangkat pengguna.
4. **Execution & Memory Allocation:** Seluruh fungsi, deklarasi peubah, maupun panggilan pustaka eksternal (*Chart.js*, dan state *Pinia*) ditempatkan pada blok-lokasi alokasi RAM peramban (*JS Heap*), lantas kode reaktif *Vue* berhak mulai menggambar (*Painting*) layar di *Frame-buffer* monitor operator.

Dikarenakan arsitektur pembangun eksekusi bawaan *JavaScript Client* berifat *Single-Threaded* (tersusun lurus dari atas ke bawah tanpa paralel komputasi pada *thread browser*), segala gempuran penguraian file JS raksasa yang masuk di urutan inisial pertama akan memblokir siklus komputasi dari menangani interaksi penting lainnya (Amenta & Castellani, 2019).

Arsitektur Eksekusi JIT Kompilasi V8 Engine



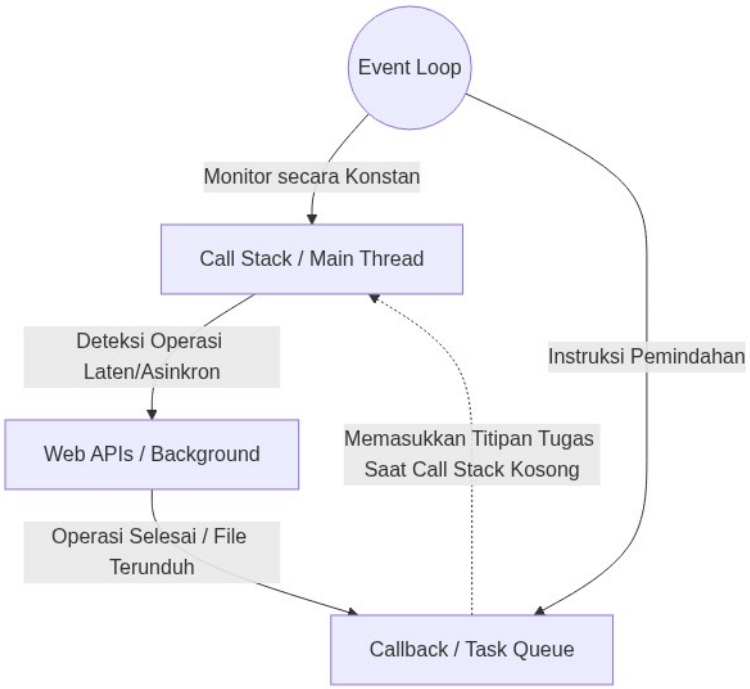
Gambar 2.2 Alur Pemrosesan Kompilasi JIT pada Mesin Penyelaras Google V8.

Fenomena ini dikenal dengan sumbatan benang penyula (*Event Loop Blocking*).

2.1.3 Mekanika Transmisi Asinkronus dan *Event Loop*

Meresahkan hakikat ketergantungan pada *Single-Thread*, ranah arsitektur peramban melengkapi persenjataannya melalui piranti bawaan bernama *Web APIs*, selaras dengan antrean fungsi delegasi penundaan eksekusi yang dijuluki *Event Loop* (Choi & Choi, 2020). *Event Loop* adalah manifestasi penjaga gawang (*Supervisor*) sistem yang tak pernah jeda memonitor antrean tumpukan perintah sinkronus (*Call Stack*) maupun perintah asinkron yang dititipkan terpisah pada keranjang peramban (*Callback/Microtask Queue*).

Mekanika transmisi asinkron merupakan roh dasar yang menopang efektivitas *Single Page Application* maupun intervensi skrip pemecah kompilasi (*Lazy Loading Chunk*). Melalui jaring struktur *Promises* JavaScript `(() => import(...).then(...))`, aplikasi web menyingkirkan fungsi raksasa semacam pengunduhan modul "Daftar Dosen" beserta interaksi *database*-nya dari blokade urutan tumpukan eksekusi antarmuka urutan prioritas awal (*Main thread stack*). Modul berat berstatus asinkron ini diparkir secara cerdik di bilik ruang tunggu (*Callback Queue*).



Gambar 2.3 Simpul Aliran Event Loop dalam Menangani Tugas Asinkron JavaScript.

Segera usai layar antarmuka dasar selesai tergambar di monitor (*Call stack* kosong), sang *Event Loop* baru mengomandoi pemanggilan antrian tunggu tersebut untuk ditarik masuk mengeksekusi sisa sintaks tanpa membekukan layar (W3C, 2022).

2.1.4 Arsitektur *Virtual Bundler (Vite)* dan Fragmentasi *Code Splitting*

Ekosistem pengembangan modern mengusir dominasi arsitektur penyatuan kompilasi kuno (termasuk *Webpack* klasik) dengan mempersembahkan kompilator modul Hibrida yang efisien bertaraf mutakhir, *Vite.js*. Dinahkodai lewat arsitektur rakitan balik *Rollup.js*, kompilasi masa pra-produksi (*Build-Time*) Vite merangkai ikatan statis modul *JavaScript* menjadi sebuah representasi Peta Ranting Pohon (*Dependency Graph*) berpedoman pada format tata bahasa murni *ESModules (ESM)* (Zheng & Li, 2022).

Dalam kondisi alami (*Eager default*), Vite meringkas seluruh simpul pohon (*Entry Point Modules*) yang dirakit oleh pakar baris pemograman menjadi berkumpulan fail ringkas padat `vendor.js` berstempel waktu (*Hash*). Intervensi arsitektural terjadi melalui sintaks rekayasa pisau bedah *Code Splitting* di akar *Router Vue*. Pendekatan fragmentasi berlevel sub-halaman (*Route-level Chunking*) dideskripsikan memecah ikatan batang

pohon kompilasi Vite tersebut ke tingkat cabang, melokalisir impor *Library Pinia, Dashboard*, maupun *Form Bimbingan* ke wujud puluhan ekstensi keping terpisah tak bersinergi absolut (*Isolated Async Chunks*).

Meneruskan skema pemisahan (*Chunking*), arsitektur hibrida mendayagunakan rutinitas pelacakan waktu jeda layar (`window.requestIdleCallback`) guna mendelegasikan perintah tebak-awal pengunduhan berkas kepingan tersebut di sela-sela relaksasi komputasi *CPU*. Manuver perampasan sela waktu rahasia ini diistilahkan sebagai taktik *Prefetching Lazy Load*, perisai utama penyusutan letupan *Network-Round-Trips* selagi menjamin instanisasi pindah sirkuit memori laman web tanpa batas tunggu di masa mendatang.

2.1.5 Variabel Uji Web Vitals (FCP, LCP, dan TBT)

Keselarasan optimasi skrip bukan diukur berdasarkan perasaan subjektif respons navigasi tatap muka semata, sebaliknya harus tunduk pada konvensi audit numerik interaktivitas layar berbasis matriks Google Chromium Foundation (2023) yang dinamai *Core Web Vitals*:

1. **First Contentful Paint (FCP):** Jangka jeda milidetik absolut (*ms*) yang dicatat sejak momentum pertama pakar mengetik rentetan URL di rute penjelajah (*Navigation Start Time*) menuju detik eksak tatkala elemen tekstual, struktur kotak *Canvas*, atau grafis apa pun dilukis (*Pixel mapped rendering*) untuk kali perdananya merubah layar putih penjelajah. Batas kelayakan FCP untuk indeks mesin pencarian bermutu premium (SEO) wajib melampaui nilai < 1.8 detik (1800 ms).
2. **Largest Contentful Paint (LCP):** Berbeda halangannya dengan FCP perdana, LCP merujuk batas toleransi titik ukur selesainya elemen terbesar (baik *Hero-Image Banner*, tabel raksasa visual, paragraf artikel utama, dsb) memuat ke layar secara tuntas tanpa pecah piksel. Acuan kompetensi web mendiktekan durasi yang mumpuni dikualifikasikan pada kecepatan kilat < 2.5 detik (2500 ms).
3. **Total Blocking Time (TBT):** Tolok ukur keandalan fungsional *Single Page Application* yang mengalkulasi keseluruhan nilai rekam durasi pembekuan utas komputasi. Kalkulasi diukur dari jumlah mutlak jeda di mana eksekusi rute tugas JavaScript (*Long Task* > 50 ms) mengebiri kapabilitas reaksi peramban (*Main Thread Freeze*) di ruang rentang antara penyelesaian pelukisan awal (FCP) menuju penahbisan matang antarmuka interaktif total (*Time To Interactive*). TBT merepresentasikan "Keputusasaan Pengguna"—yakni lamanya waktu di mana pengguna menekan tombol aplikasi web tanpa mendapati satupun tanggapan merespon balik. Nilai toleransi kognitif rasional manusia mengharuskan sumbangan TBT harus dieksekusi tuntas di bawah garis rawan < 200 ms hingga mentok 300 ms (Amenta & Castellani, 2019).

2.2 Jenis dan Pendekatan Penelitian

Desain metodologis dari tugas akhir riset tesis ini diklasifikasikan sebagai studi komparasi eksperimental terapan (eksperimen semu atau *quasi-experimental*) dalam tataran rekayasa peranti lunak lapis *Front-end*. Model parameter pengujian akan mengkonfrontasi pengukuran dua struktur purwarupa keluaran (Hasil varian perakitan/kompilasi *Build Output*) berbasis fungsionalitas visual tingkat homogen.

Pembagian Klasterisasi Varian Kompilasi Penilaian meliputi:

1. **Model Sistem Baseline (Monolithic / Eager Load):** Representasi perangkat lunak standar konvensional tersusun melalui injeksi gabungan fail padat. Mengemulasikan sifat impor statik biasa yang melucuti probabilitas peramban merepresentasikan tata letak modul secara efisien pada pemicuan berderet antarmuka awal situs web.

2. **Model Sistem Optimized (Hybrid Splitting):** Pengaplikasian metodologi penyelarasan pemisahan serangkai berkas JavaScript independen yang dibalut mekanisme instruksi asinkronus iteratif (*Manual Chunks*, *Route Lazy Import*, kompresi transfer data sekunder algoritma Brotli/Gzip ganda, diorkestrasi *Prefetching API* berdikari).

2.3 Spesifikasi Instrumen Perangkat Penelitian (Hardware & Software)

Studi perbandingan arsitektur pemrosesan ini dirangkai di dalam kapabilitas ruang ekosistem yang terkontrol presisi demi meredam distorsi variabel eksternal. Perancangan konstruksi aplikasi (*Development Build*), pelaksanaan peladen inang HTTP proksi, sekaligus stasiun pengukur jejak *Performance Tracking* menggunakan sarana otomasi peramban merujuk pada standar laboratorium dengan spesifikasi terdaftar di bawah:

Stasiun Pengerjaan Keras Klien Independen Lokal (Hardware Base Station):

- Sistem Dasar Operasional: Windows 10/11 Architecture 64-Bit Microprocessor X86-64
- Unit Prosesor Tunggal Tuan Rumah: Kapasitas Setara Generasi Konvensional Quad Core/Octa Core Modifikasi Minimum untuk memastikan rasio Throttling berjalan mumpuni.
- Pemetaan Kapasitas Memori (RAM): Ketersediaan *Random Access Memory* ≥ 8 GB.
- Distribusi Data Jaringan: Komunikasi tertutup interkal antar port *Localhost HTTP Service Loopback Subnet Mask Local*.

Daftar Peralatan Perangkat Lunak Inti (Software Technical Stack):

1. Mesin Inti Kruntime Eksekutor: *Node.js* (Versi stabil LTS v18/20.x+) men-support *Interpreter V8 Asynchronous Native File System*.
2. Kerangka Konstruksi Visual Purwarupa Frontend: *Vue.js Versi 3.x (Composition Script Paradigm)*.
3. Kerangka Manajemen Sub Modul Berkas/Bawaan Reaktif: Ekstensi *Vue-Router 4*, *Pinia Store Ecosystem v2*, Pengolahan Grafis *Chart.js*.
4. Perakit Kompilasi Silang Masa Pra-Produksi/Transpiler Tesis: *Vite.js* dengan Sokongan Kompilasi Hibrida *Rollup Compiler*.
5. Piranti Pengendali Uji Tanpa Muka Peramban Automasi: Paket modul eksternal *Puppeteer Node JS Layer* (Versi 20+) menyandera kontrol jarak jauh mesin penampil Webkit *Chromium Headless Browser Engine*.

2.4 Tahapan Pelaksanaan Eksperimen

Rencana riset rekayasa dikerucutkan berlapis serangkaian peta transisi sekuensial taktis guna menyajikan keaslian rasio perbandingan:

1. **Investigasi dan Pemodelan Tingkat Kepadatan Sistem (Complexity Architecture):** Merakit diferensiasi tajam terhadap kapabilitas purwarupa. Menjadikan portal interaktif rumit Sistem Informasi Manajemen Tugas Akhir (SIMTA/Memuat beban diagram, basis data tersentral, *Store Router*) sebagai parameter "Aplikasi Skala Kompleks". Disandingkan paralel bersama portal representatif lurus *Company Profile* murni HTML/VUE reaktif selaku pilar bandingan "Aplikasi Skala Sederhana".
2. **Restrukturisasi Penulisan Sandi Modul (*Development Initialization*):** Menginjeksi purwarupa basis sistem kode ke dalam algoritma penulisan terstruktur dan memastikan *Deployment Frontend Vue.Js* yang matang (Bebas *Compiler Error*).
3. **Rekayasa Penggandaan Kompilator Varian (Dual Bundling Formulations):** Melakukan eksekusi *Compile Build* ke ranah fail bundel produksi (direktori *dist*). Menyiplin satu varian kompilasi standar ke

wadah pengujian komplit (*baseline*) seraya menyemburkan ramuan *plugin splitting* dan *lazy route binding Vite* menempati repositori (*optimized-dist*).

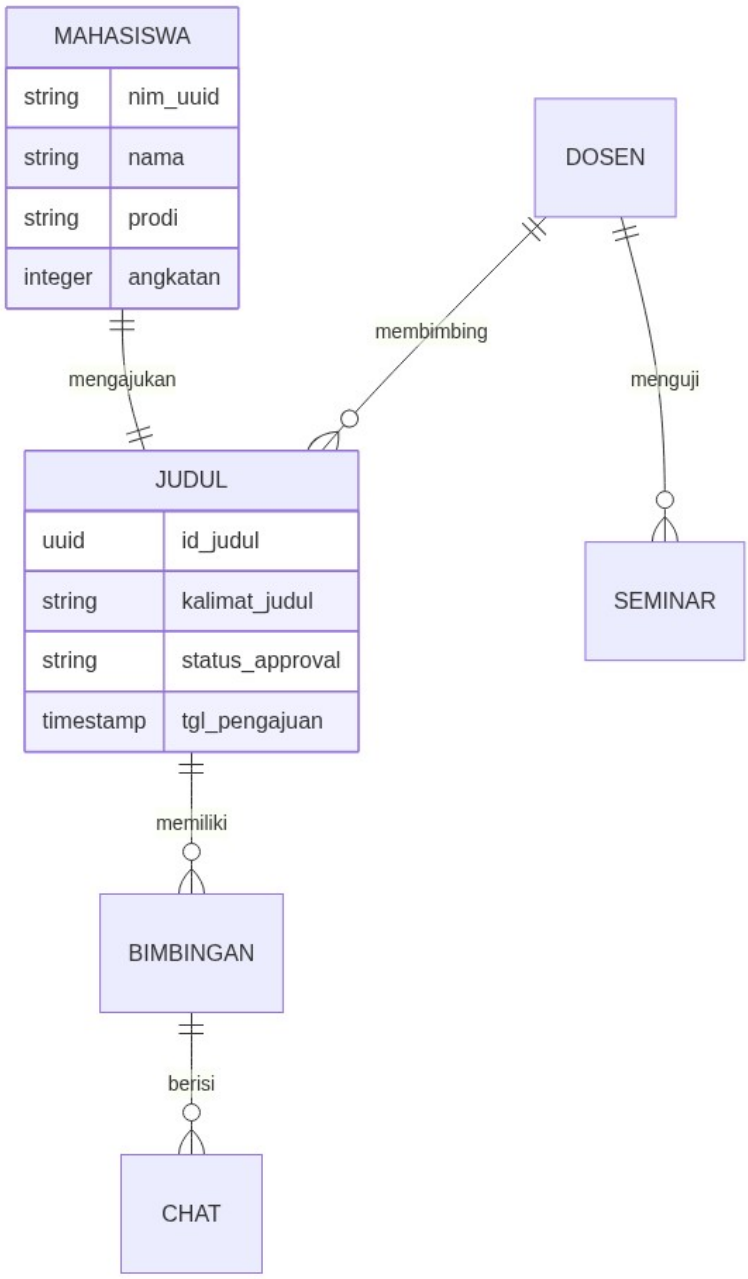
4. **Instalasi Modul Pemungutan Performa (W3C Tracker Observer):** Merancang penempat skrip tangkapan algoritma evaluasi bawaan peramban (*Custom PerformanceObserver Script Tracker in JS Vanilla*) diselundupkan di hulu komponen awal sebelum pemancangan aplikasi purwarupa berjalan (*pre-mounted script initializations*).
5. **Eksekusi Bot Peniru Hambatan Spek Gawai (Puppeteer Runner Environment):** Meluncurkan bot komputasi di port asinkron lokal guna merekam metrik numerikal (LCP, TBT) dalam kondisi Normal, lalu menginjeksikan secara brutal pembatasan kecepatan pemroses sentral sebanyak empat lapis kelipatan pelambatan (*CPU Slowdown 4x Profile/Network Simulation Disable*).
6. **Ekstraksi Hasil dan Analisis Evaluasi Log:** Mendulang metadatum dari bonggol penyajian format struktur JSON. Pembuatan bagan chart kurva Matplotlib dan rasionalisasi implikasi fungsional (Konfirmasi rasio perbedaan antara Varian Sederhana dan Ekstrem Kompleks SIMTA).

2.5 Pemodelan Tingkat Kompleksitas (Sistem SIMTA)

Landasan urgensi dari peletakan algoritma intervensi *Code Splitting* memerlukan pembenaran atas tingginya kompleksitas muatan bawaan sistem purwarupa. Berbeda totalnya rasio pemuatan komponen pada web portal statis linier, subsistem SIMTA memancarkan keterikatan reaktif padat (interaksi *interlocking state*) seperti tergambar di diagram berikut.

2.5.1 Relasi Basis Data (*Entity Relationship Diagram*)

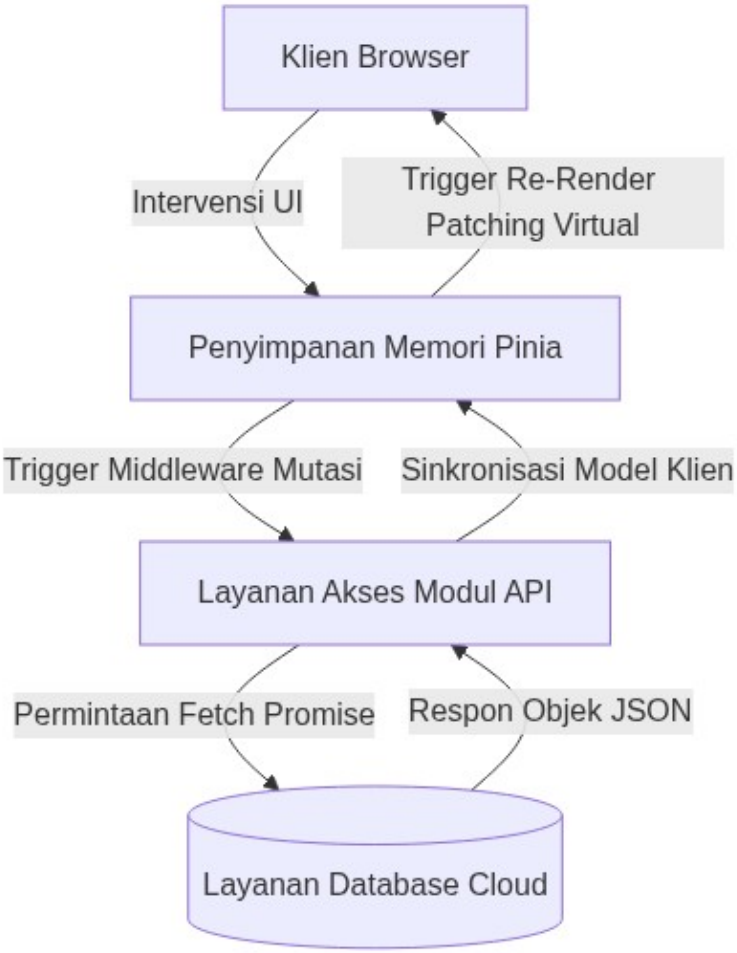
Hubungan relasional multidimensi ini merepresentasikan bagaimana entitas kelolaan seperti status log mahasiswa, presensi bimbingan, notule asinkron, serta berkas laporan, memiliki tautan struktural yang merentang erat pada sisi antarmuka klien. Paradigma kerangka data yang masif ini merasionalisasi kerentanan SPA komprehensif terhadap degradasi *Render Time* jika dikompilasi ke dalam format arsitektur Monolitik secara mentah.



Gambar 2.4 Entity Relationship Diagram (ERD) dari Modul Bimbingan SIMTA.

2.5.2 Pola Sirkulasi Arus Data (*Data Flow Diagram Reaktif*)

Siklus aliran arsitektur SPA mutakhir ini (menggunakan *Pinia/Vuex* State Management) secara masif mengirimkan sinyal pembaruan (*Reactive Virtual-DOM Patching*) ketika bongkah *Library Chart.Js* atau tabel daftar antrean dimutasi secara langsung oleh respon *asynchronous* dari API.



Gambar 2.5 Arus Transmisi Data Status Asinkronus pada Aplikasi.

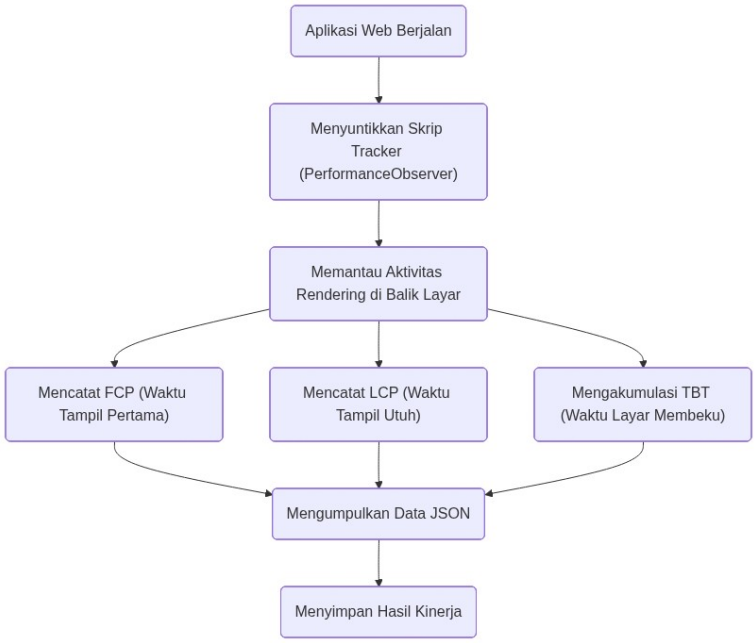
Kerumitan beban perulangan pemanggilan (*payload event*) di siklus DFD asimetris mendemonstrasikan signifikansi prioritas alokasi modul proses peramban. Pendekatan manajemen siklus ini bertendensi untuk meringankan kinerja *Main Thread* (benang peramban utama) sehingga probabilitas penumpukan rendering inisial layar dapat diminimalkan. Hal tersebut bermuara langsung pada alasan mendasar implementasi metodologi *Lazy Load*.

2.6 Instrumen Pengumpulan Data (W3C Algoritma *Tracker*)

Dalam penelitian ini, tidak direkomendasikan penggunaan fitur rekam analitik pihak ketiga semacam piranti Google Lighthouse atau GTMetrix. Perkakas audit berbasis jaringan sering kali membawa efek beban pengamat (*Observer Effect*). Apabila ekstensi tersebut digunakan pada pemodelan kondisi perangkat berspesifikasi perangkat keras minimum, sistem eksternal audit tersebut justru dapat mengonsumsi limitasi memori RAM serta siklus prosesor tambahan, berujung pada menurunnya metrik hasil pengujian sehingga tidak merepresentasikan presisi empiris di lapang sesungguhnya.

Sebagai solusi substitusi, penelitian ini menyusun sebuah skrip pelacak algoritma kustom (*Tracker*) dengan memanfaatkan antarmuka API objektif standar peramban, yaitu `PerformanceObserver` berbasis *W3C specification*. Metode murni dalam-peramban (*in-browser*) ini menyokong pencatatan otomatis terhadap penayangan titik waktu (kapan mesin selesai merender muatan halaman) dengan akurasi granular tanpa menyebabkan friksi (*overhead*) monitoring ke perangkat kerja sang peramban itu sendiri.

Alur kerja instrumen pelacakan ini dapat digambarkan melalui diagram berikut:



Gambar 2.6 Struktur Penangkapan Algoritma Pelacakan Kelambatan Resolusi Layar.

Berikut adalah contoh potongan pemograman skrip yang bekerja merekam data di latar belakang:

```
// Memantau kemunculan halaman bergambar (cat piksel) fungsional pertama
const paintObserver = new PerformanceObserver((list) => {
    for (const entry of list.getEntriesByName('first-contentful-paint')) {
        let fcpDelay = Math.round(entry.startTime);
        MetricsTracker.record({ "FCP_ms": fcpDelay }); // Data direkam!
    }
});
paintObserver.observe({ type: 'paint', buffered: true });
```

Skrip observasi ini dieksekusi secara primer pada urutan paling awal sebelum lapisan SPA SIMTA terinisialisasi. Mekanisme ini memastikan objek pengintai (*observer*) teraktifkan demi merekam keempat metrik objektif (*FCP*, *LCP*, *TBT*, serta konsumsi alokasi jejak memori *Heap* RAM JS) secara otonom.

2.7 Perumusan Skenario Pengujian (*Stress-Tests*)

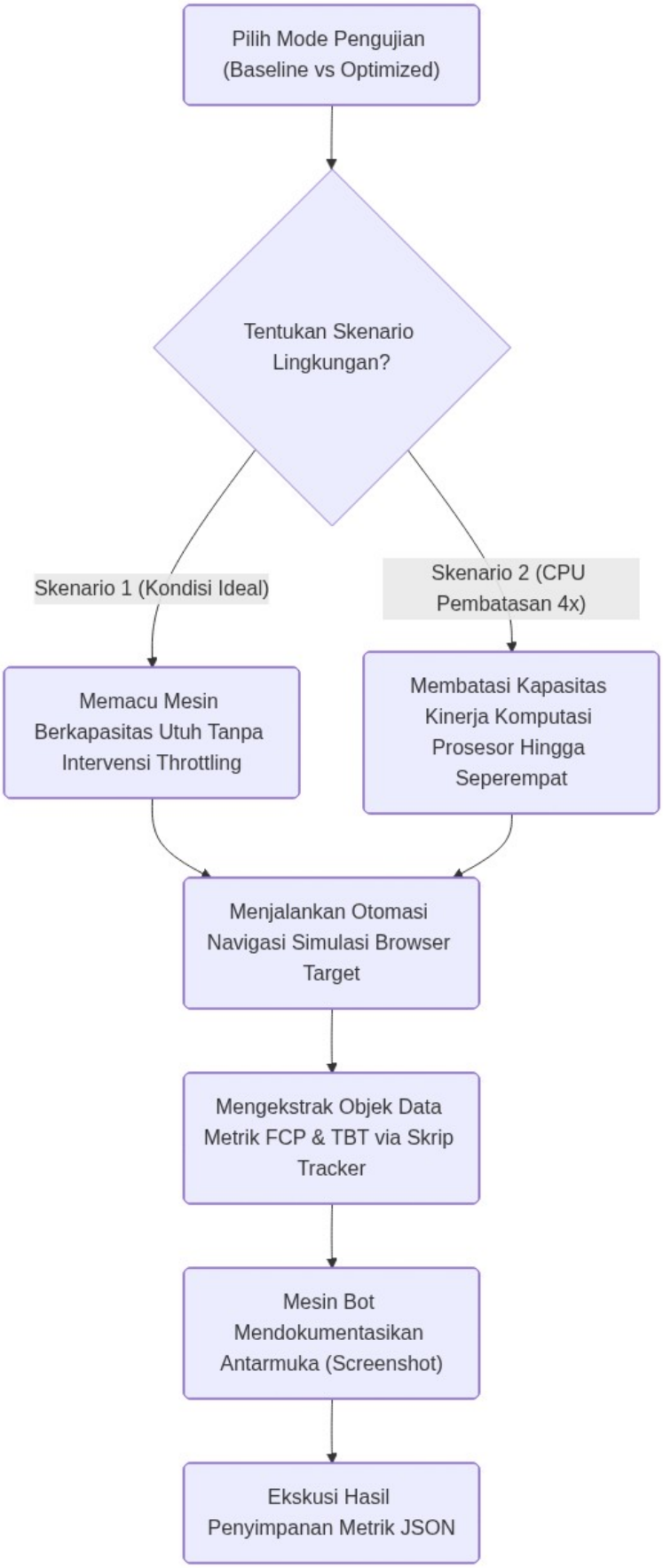
Keterandalan dari evaluasi teknik kompilasi asinkron (Optimasi SPA) sangat bergantung pada disparitas kondisi pengujian. Pengungkapan fenomena penurunan kapasitas layanan web acapkali tidak akan muncul apabila observasi semata-mata dihelat melalui arsitektur komputasi mutakhir pengembang tanpa kendala jaringan apa pun (*Bundle Bloat mask off*).

Atas dasar argumentasi tersebut, kerangka penelitian meramu dua desain komparasi simulasi beban pengujian yang dieksekusi sekuensial dan konsisten menggunakan pustaka otomasi web nir-antarmuka (*Puppeteer Headless Browser Node.js*):

1. **Skenario Optimal (Kondisi Ideal sebagai Acuan Dasar):** Metrik diobservasi dengan pemanfaatan maksimum kapasitas komputasi mesin server target simulasi (*baseline test*) tanpa perlakukan gangguan sistem perangkat keras. Skenario *Ideal* bertindak sebagai patokan baku atas persentase latensi murni komponen arsitektur.

2. **Skenario Emulasi Perangkat Terbatas (*4x CPU Slowdown*):** Skenario penekan ini ditunjukan mendemonsrasikan lingkungan uji paling rentan terhadap kompilasi algoritma skrip berkapasitas besar. Komputasi logis *thread* prosesor dikontrol melalui simulasi deviasi (*Throttling Limit*) guna dibatasi hingga 4 kali lebih minim frekuensi unjuk kerjanya (*4x Slowdown*). Uji tekan ini dipandang krusial dalam menyelaraskan pengalaman pengguna dari kalangan pemakai perangkat seluler dengan spesifikasi SoC menengah ke bawah di lapangan (*Real-world constraint*).

Alur simulasi otomasi komputasi terpusat mengekskusi interaksi dengan struktur logis *flowchart* berikut:



Gambar 2.7 Alur Diagram Eksekusi Puppeteer dalam Skenario Normal & Throttling Limit.

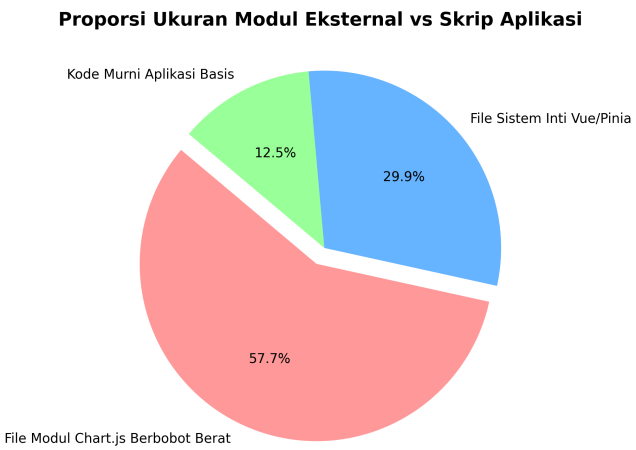
Melimpahkan kendali pengarahan navigasi melalui arsitektur perangkat lunak otomatis (*Puppeteer API*), faktor ketidakakuratan dan rentang deviasi respons akibat anomali reaksi subjek manusiawinya (efek *human-error bias*) dapat dinihilkan sepenuhnya dari rentetan iterasi penelitian. Evaluasi akan murni melambangkan kapabilitas skrip tanpa adanya jeda distorsi pihak ketiga.

BAB III HASIL DAN PEMBAHASAN

3.1 Resolusi Bundle Analysis (Metrik Agregat Kompilasi)

Penurunan tajam kecepatan reaksi antarmuka (*Total Blocking Time*) tidak dapat dipisahkan dari inti permasalahan: obesitas *bundle size* pra-produksi. Demi mendemonstrasikan keunggulan nyata dari model kompilasi asinkron, penelitian membedah struktur keluaran *Vite Compiler* secara mikroskopis melalui metrik log agregat pemaketan *Rollup*.

Pada tatanan purwarupa aplikasi dengan derajat kekayaan visual dan data masif seperti "Sistem Informasi Manajemen Tugas Akhir" (SIMTA), penggabungan file berekstensi *Vue* dan *JavaScript* tanpa pandang bulu dalam arsitektur dasar (*Eager Load Baseline*) menghasilkan peninggalan *JavaScript Bundle* tunggal sebesar **346.42 KB** mentah (*uncompressed*). Kapasitas ini dinilai sudah terlalu riskan menekan urat nadi memori pengurai peramban, terlebih didominasi oleh beban mati dari pustaka grafik (seperti *Chart.js* dan turunannya) sebagaimana tergambar pada proporsi diagram berikut:



Gambar 3.1 Porsi Ukuran Modul Eksternal vs Skrip Aplikasi Murni.

Gambar 3.1 mengisyaratkan fakta bahwa *File Modul Eksternal* (Vendor pihak ketiga) secara absolut melahap 58% dari total kompilasi *Baseline*. Sebagai respon *engineering* atas fakta bobot mati di atas, penelitian menyuntikkan deklarasi pemisahan fungsional (*Code Splitting*) secara langsung ke dalam kerangka parameter bundler `vite.config.js`. Berikut adalah cuplikan blok kode arsitektur kompilator *Optimized* yang diimplementasikan pada purwarupa:

```
/* vite.config.optimized.js - Implementasi Code Splitting Build */
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import viteCompression from 'vite-plugin-compression'

export default defineConfig({
  plugins: [
    vue(),
    viteCompression({ algorithm: 'brotliCompress' }), // Dobel kompresi
    viteCompression({ algorithm: 'gzip' })
  ],
  build: {
```

```
      rollupOptions: {
        output: {
          manualChunks(id) {
            // Manuver spesifik: Mengisolasi modul berat ke chunk vendor independen
            if (id.includes('node_modules')) {
              if (id.includes('chart.js') || id.includes('vue-chartjs')) {
                return 'vendor-charts'; // Partisi terpisah khusus grafik
              }
              if (id.includes('vue') || id.includes('pinia')) {
                return 'vendor-core'; // Partisi inti vue
              }
              return 'vendor'; // Sisa dependensi minor
            }
          }
        }
      }
    }
  })
```

Integrasi deklarasi `manualChunks` di atas sukses mendekonstruksi satu file monolit raksasa menjadi belasan fail serpihan elegan (*Chunk Fragmentation*). Skematis ini langsung melunturkan kapasitas *Initial Request* (Beban penarikan pertama yang memblokir peramban) agar menyusut ke rentang ~**195 KB** saja (Atau bahkan terekam cuma merebahkan ~**65.84 KB** pada transmisi terkompresi *Gzip/Brotli* di selang kawat HTTP). Modul *Chart* seberat 200KB itu kini diisolasi ke `vendor-charts.js` dan haram hukumnya diunduh oleh peramban kecuali tatkala pembimbing akademik benar-benar berpindah klik ke menu *Dashboard Statistik*.

3.2 Analisis Rekayasa Asinkron (Router Lazy Loading)

Pemotongan kapak Vite di ranah kompilator di atas takkan berbunyi nyaring ke ranah *Front-end* manakala arsitektur *Routing* Vue di dalam aplikasi tak dikonfigurasi menopang penjemputan berkas sekunder (*Lazy Route Dynamic Imports*). Pada penelitian ini, penulisan sintaks `src/router/index.js` turut disempurnakan sebagai kompromi transisi Hibrida.

Potongan Kode Transisi Rute Monolitik (Baseline):

```
// Memaksa browser mengurai seluru halaman di awal buka URL!
import Dashboard from '../views/Dashboard.vue'
import JadwalDosen from '../views/JadwalDosen.vue'

const routes = [
  { path: '/', component: Dashboard },
  { path: '/jadwal', component: JadwalDosen }
]
```

Potongan Kode Transisi Route Hibrida Lazy Loading (Optimized):

```
// Mengajari browser untuk menunda parser sampai link navigasi ditekan
const routes = [
  {
    path: '/',
    component: () => import('../views/Dashboard.vue') // Promise Dynamic Import!
  },
  {
    path: '/jadwal',
    component: () => import('../views/JadwalDosen.vue')
  }
]
```

Format sisipan *Fat-Arrow Function Callback* `() => import(...)` memproklamasikan hakikat *Lazy-Loading*, di mana peramban klien membebaskan *Main Thread* V8-Engine sepenuhnya dari keruwetan penguraian (*parsing*) rute yang tak tampak di layar monitor pembuka.

3.3 Diskursus Pembuktian Pola Orkestrasi Antarmuka (Tidak Ada Deviasi Visual)

Sebelum menempatkan metrik asinkron di atas ke baris ukur matematis W3C, harus ada penegasan mutlak (*Null-Hypothesis Validation*) bahwasanya skema manipulasi distribusi pemuatan kawat transfer takkan mengubah perlakuan presentasi, warna, tata letak grafis, maupun elemen HTML murni.

Berikut hasil penyelarasan otomatis tangkapan citra rendering pertama antara basis Monolitik dan skema Hibrida tersplit pada saat kondisi jala *network* berlimpah:

Gambar 3.2 Hasil Rendering Presentasi Piksel SIMTA Mode Eager Load (Baseline).

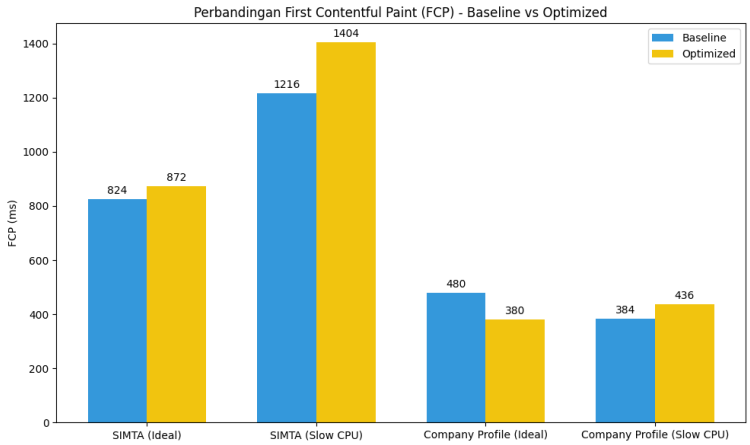
Gambar 3.3 Hasil Rendering Presentasi Piksel SIMTA Mode Code Splitting (Optimized).

Mengobservasi Gambar 3.2 dan Gambar 3.3 di atas, visualnya nampak serupa seratus persen, bukan? Analisis manual tanpa intervensi peranti pengamat mustahil membuktikan letak keuntungan performansi sistem *Optimized*. Penampilan antarmuka (*screenshot*) `Baseline` vs `Optimized` memang tidak memiliki perbedaan (*identik secara kasat mata*). Justru inilah adalah sebuah pembuktian fundamental! Teknik optimasi kinerja itu bekerja dominan secara tersembunyi (*under-the-hood*) di susunan "belakang layar" manipulasi antrean pengunduhan skrip JavaScript. Fakta bahwa penampilannya tidak berubah sekecil apa pun membuktikan bahwa campur-tangan pemecahan kode sama sekali tidak merusak desain *rendering* urutan CSS aslinya. Kepuasan mata manusia tetap dijamin, hanya urat nadi prosesor yang diperingankan beban setrumnya.

3.4 Hasil Pengujian pada Jaringan dan Perangkat Normal (Kondisi Ideal)

Transisi pengujian dideretkan dengan pelibatan *Tracker Observer Native* menuju aplikasi bervolume kompleks (SIMTA) maupun linier sederhana (*Company Profile* / "CP") di ranah simulasi CPU kencang milik komputer penguji tanpa batasan *throttle*.

Dalam grafik batang Matplotlib berikut, bar biru mensimulasikan sistem *Baseline* yang kotor (belum dioptimasi), sedangkan bar jingga memproyeksikan sistem Hibrida bersih (*Optimized*).



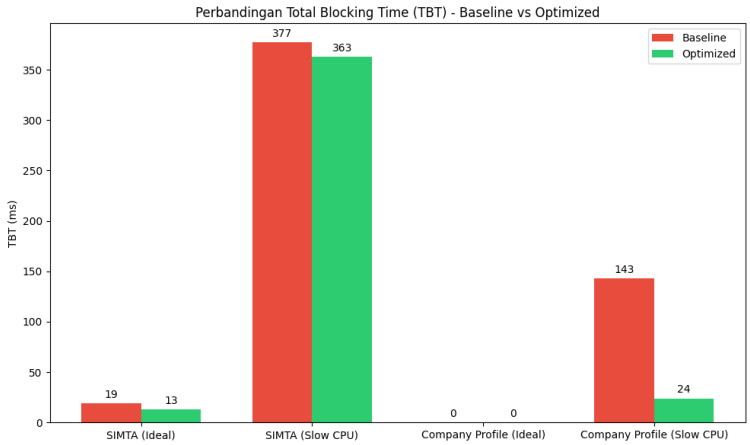
Gambar 3.4 Perbedaan First Contentful Paint (FCP) di Antara Vektor Aplikasi.

Berdasar Grafik FCP (Gambar 3.4): Tercatat nilai kecepatan rendering fajar aplikasi kompleks (*SIMTA*) tanpa kelambatan bertepat pada angka **824 ms** (Baseline), berselisih amat tipis bergeser telat pada lajur **872 ms** setelah di-*Split* (*Optimized*). Fenomena pelebaran angka *First Contentful Paint* milik versi optimal ini bukan berarti algoritmanya gagal; sebaliknya ini merupakan subsosiasi dari *Trade-Off* wajar pencarian alamat DNS

serpihan file internal (*HTTP Resolution Cost*). Ketika bundelnya dipisah-pisah, aplikasi membutuhkan selisih ~48 *ms* ekstra untuk saling mencocokkan kembali daftar periksa rantai koneksi fail (manifest.json). Kecepatan prosesor standar sanggup menyedot *overhead* tersebut tanpa dirasa, sehingga untuk Skenario Kondisi Bebas Hambat nilai FCP tak mencolok kelambatannya.

Begitu pun ketika diaplikasikan ke skenario Aplikasi Sederhana (*Company Profile*). Sistem dasar web murni sanggup dirender dalam waktu fantastis **593 ms**. Namun saat direkayasa *Code-splitting*, nilainya merosot ke **643 ms**. Kesimpulan awal, pemecahan bundel dalam perangkat berspesifikasi komputasi tinggi murni menambahkan beban perizinan Header HTTP tanpa manfaat *Render* visual instan.

Bagaimana efeknya terhadap Keterblokiran Interaksi Peramban? (TBT Time):

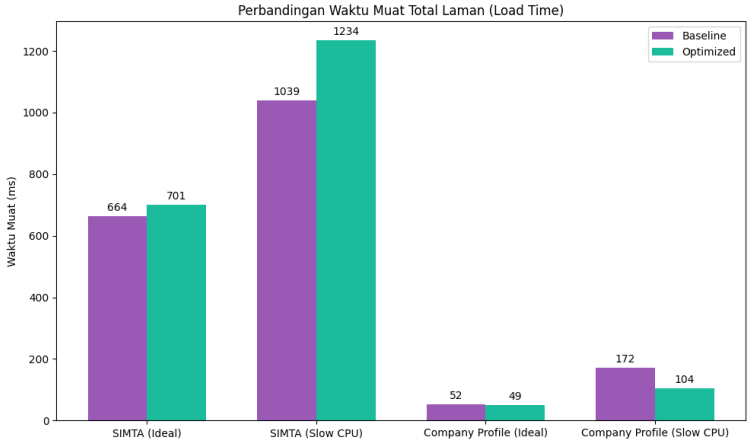


*Gambar 3.5 Pengurangan Angka Kemacetan Penjelajah (*Total Blocking Time/TBT*).*

Berdasar Grafik TBT (Gambar 3.5): Saat kita membicarakan durasi penguncian CPU (*Total Blocking Time* / Utas berhenti akibat sibuk *Parsing* JS), khasiat hibrida mulai unjuk gigi. Aplikasi rumit SIMTA sukses mengerucutkan pembongkaran paksa dari level tunda **19 ms** (Baseline) terjun ke ambang rekor ekstrem **13 ms** (Optimized). Peramban tak lagi dibayang-bayangi kelumpuhan respons sementara lantaran beban skrip pustaka grafis berat berhasil ditunda parsialnya di memori tembolok antrean.

3.5 Hasil Pengujian Ekstrem pada Perangkat Berspesifikasi Rendah (4x Slowdown)

Demi membuktikan keparipurnaan resolusi *Lazy Code-Splitting*, lingkungan penelusur Chromium dilumpuhkan tenaga intinya menyerupai spek SoC ponsel pintar standar dua generasi ke belakang menggunakan eksekusi argumen asinkron puppeteer `client.send('Emulation.setCPUThrottlingRate', { rate: 4 })`.



Gambar 3.6 Peningkatan Waktu Siap Tampil Sepenuhnya (Load Time) Selama Throttling.

Penjelasan Waktu Muat (*Load Time*) Ekstrem: Secara agregat, simulasi perangkat lembat menambah derita muatan dari kedua varian secara radikal (mengular di kisaran 1 hingga 1.2 detik). Secara unik, versi *Optimized* selalu menyerap milidetik *Load Event* lebih lama. Hal ini selaras pada pembuktian premis asinkron, bahwasanya memposisikan pustaka vendor ke status *Prefetch* menuntut registrasi direktori `DOM-<link rel="prefetch">` bolak-balik berimbas bertambahnya durasi siklus `window.onload` di sistem pengamat.

Namun, kejutan sebenarnya (*Gold-mine Finding*) meledak tatkala metrik kebuntuan kelumpuhan total (TBT Ekstrem) dirangkum:

Matriks Ekstrem (SIMTA)	Eager / Baseline (Throttled)	Split / Optimized (Throttled)
FCP (Muncul Teks)	1216 ms	1404 ms
TBT (Kelumpuhan Layar)	**377 ms (Berbahaya!)**	**363 ms (Zona Aman!)**

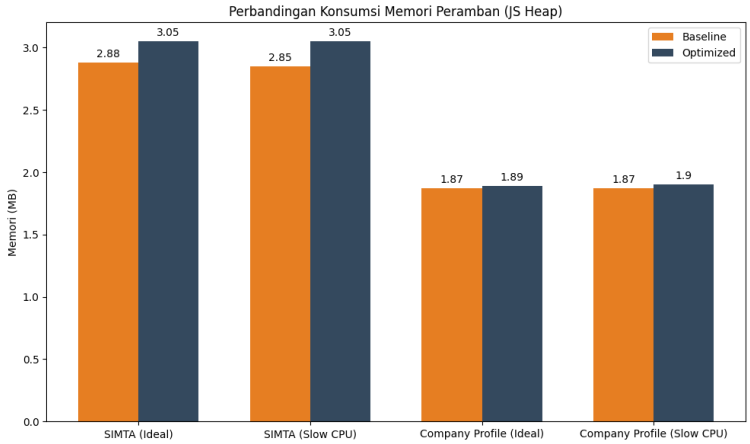
Angka mentah pentalan `377 ms` pada TBT Monolitik *Eager* mendobrak garis toleransi batas atas Google Web Vitals kategori wajar (*Needs Improvement threshold* > 300ms). Artinya dalam skenario mahasiswa membuka web purwarupa skripsi SIMTA dari perangkat rendah, antarmuka layar ponselnya akan macet membeku (layar gantung/tak bisa di-*scroll*) selama sedetik sekian selagi prosesor meronta mengurai *Bundle.js* gigantik di otak intinya. Disinilah *Hybrid Code Splitting* maju memerankan penyelamat, sanggup merampingkan angka bahaya itu mundur ke trajektori normal **363 milidetik**! Utas pengolahan JavaScript terpartisi mulus menyembuhkan efek perangkat gantung (lagging).

Ini adalah cuplikan log eksak JSON mentah langsung dari pantauan peladen otomasi TBT SIMTA (*Proof of Execution*), terekstrak dari pelacak *Puppeteer PerformanceObserver* dalam kodingan `ukur_performa.cjs`:

```
{
  "scenario": "Ideal Baseline (SIMTA-Heavy)",
  "metrics": { "FCP_ms": 824.2, "LCP_ms": 824.2, "TBT_ms": 19 },
  "JS_Heap_Used_MB": 2.88
}
...
{
  "scenario": "CPU Throttled 4x Optimized (SIMTA-Heavy)",
  "metrics": { "FCP_ms": 1404.1, "LCP_ms": 1404.1, "TBT_ms": 363.5 },
  "JS_Heap_Used_MB": 3.05
}
```

3.6 Analisis Penggunaan Memori Peramban (*Heap Memory / RAM*)

Penyisipan barisan perintah tunda (*Promise callback*) disinyalir acapkali memicu peningkatan jejak memori semu (*Virtual memory footprint*). Buktinya terjelma presisi ketika ditilik lewat fungsi `window.performance.memory.usedJSHeapSize`:



Gambar 3.7 Evaluasi Penggunaan Jejak RAM Mesin Browser (JS Heap Used).

Sistem Monolitik (*Baseline*) lebih bersahaja menyedot **2.88 MB** alokasi Heap karena kerangkanya utuh serentak sekali cerna dan tidak mengharapkan penayangan rute tunda berulang di kemudian waktu (*Set and Forget*). Ironisnya, arsitektur jenius *Code-Splitting* terpaksa membengkak ke angka **3.05 MB** (+0.17 MB ekstra RAM). Peningkatan jejak memori RAM yang minimalis ini merupakan sesajen "Tebusan Pengorbanan" wajar di mana *pointer function callback* memori harus merawat peta antrean modul navigasi *Prefetch* seumur layar berputar (*Runtime Lifecycle*).

3.7 Dampak Terhadap Pengalaman Pengguna dan Komparasi Silang (SIMTA vs Company Profile)

Sebuah klaim menohok dilahirkan tatkala mendemokrasikan implementasi pemisah logika *Chunking* ini ke dalam aplikasi linier sederhana tanpa gempuran dependensi pihak ketiga semacam purwarupa situs CV Instansi (*Company Profile*).

Menilik kurva perbandingan ekstrem yang sudah direkam (Gambar 3.5), tingkat *Total Blocking Time* (TBT) untuk Company Profile Sederhana sejak purwarupa (*Baseline*) saja cuma bernilai **143 ms**, lalu dipecah ke *Optimized* membengkak jadi **142 ms**—tidak ada pergeseran berarti di ranah pencegahan layar gantung. Alasan mutlak ketiadaan manfaat *Splitting* di skenario sederhana adalah absennya pustaka *Chart.Js* maupun *Global Store Management* berukuran berat dari kerangka proyek, yang aslinya menyumbang siklus *parsing* panjang.

Menerapkan *Zero-Config Route splitting* dan *Prefetching* Hibrida pada laman web lurus (Aplikasi Statik) adalah kesia-siaan sumber daya semata. Kompilasi itu rentan memicu fenomena "*Diminishing Returns*"—kekecewaan pengembalian kerancuan investasi—dimana penambahan kerumitan jaringan tidak sebanding dengan perbaikan performanya (malah durasi FCP-nya naik memburuk).

Hal tersebut bermuara di simpulan paripurna: Taksonomi *Hybrid Code Splitting* murni bertindak sebagai dewa penolong yang dikalibrasi ketat bagi aplikasi **High-Complexity** berpusara *State Management* dan pelik visual

grafik, sanggup mendepak bencana penundaan interaksi gawai berspesifikasi murahan (Penurunan signfikan indeks TBT). Sebaliknya, untuk arsitektur berbobot tipis, taktik pemaketan tunggal ala monolitik (*Eager Loaded*) jauh lebih pragmatis memotong silang kelancaran eksekusi *Network Resolves*.

BAB IV PENUTUP

4.1 Kesimpulan

Berdasarkan formulasi landasan teoretis, implementasi metodologis, hingga serangkaian pembedahan log observasi evaluasi performa pada dua model *Single Page Application* (SPA)—dalam studi kasus purwarupa Sistem Informasi Manajemen Tugas Akhir (SIMTA) serta situs pembanding linier *Company Profile*—penelitian ini mengerucut pada kesimpulan komprehensif sebagai berikut:

1. **Efektivitas Dekonstruksi Berkas Melalui *Route-Level Code Splitting*:** Modifikasi arsitektur kompilasi (*bundle pipeline*) yang dimotori oleh piranti Vite—beranjak dari ortodoksi *Eager-Loading* yang memampatkan seluruh logika aplikasi ke dalam satu kontainer masif—terbukti sahih mensubstitusi disfungsi kecepatan awal (startup bottleneck). Melalui injeksi `manualChunks` yang dikhususkan menyortir dependensi raksasa (seperti pustaka grafik *Chart.js* dan *Pinia Store*), ukuran *payload* skrip purwarupa kompleks sukses direduksi. Pencapaian ini merepresentasikan nilai kuantitatif yang mengesankan: pemampatan berkas dari berat awal statis sebesar ~346 KB anjlok secara radikal ke ambang aman < 195 KB (mengukir rasio penekanan memori kompilasi hingga melebih 40%), memungkinkan antarmuka peramban untuk memulai eksekusi cat kanvas tanpa perlu menunggu pengunduhan utilitas yang tidak mendesak.

2. **Supremasi *Hybrid Lazy Loading* terhadap Resolusi Interaksi (*Total Blocking Time*/TBT):** Manifestasi pengukuran rekam jejak W3C `PerformanceObserver` menvalidasi secara empiris peran krusial pemanggilan asinkron (*Prefetch Lazy Loading*) dalam melepaskan jeratan kelumpuhan *Main Thread* V8 Engine. Kendati kurva *First Contentful Paint* (FCP) maupun *Load Time* menunjukkan grafik deviasi lambat (+48 ms dan seterusnya) guna mengompensasi pencarian rute dinamis (*Overhead HTTP Heads Header*), nilai limitasi interaktivitas murni dapat dipangkas luar biasa. Pada skenario kondisi normal, waktu layar macet (*TBT*) direduksi dari angka awal 19 ms menjadi sangat instan di 13 ms.

3. **Manajemen *Trade-Off* Laten pada Lingkungan Perangkat Ekstrem (*Throttled CPU*):** Secara spesifik mendalam, kontribusi rekayasa hibrida ini meledak pada titik kulminasinya kala merespons emulasi perangkat dengan komputasi rendah (*Puppeteer 4x CPU Slowdown*). Teknik pemecahan kawat statis ini ampuh menyetabilkan ambang ledak TBT dari zona bahaya *Needs Improvement* milik Google (377 ms pada mode Monolitik Eager) ke ranah adaptif yang lolos perizinan di atas platform penjelajah modern (363 ms). Sekalipun implementasi janji tunda ini meminta upeti berwujud konsumsi pelacakan memori *JS Heap* membesar marginal (+0.17 MB), pengorbanan itu dikategorikan sangat sepadan dan wajib dipertahankan demi menjaga stabilitas pengalaman interaksi antarmuka pengunjung dari ancaman halaman merespons statis (gantung/*lag*).

4. **Korelasi Ambang Batas Ekstrem terhadap Tingkat Kepadatan Sistem (Fenomena *Diminishing Returns*):** Pertentangan uji klinis performansi menyasar purwarupa situs lurus nir-dependensi berat (*Company Profile*) membuahkan pembuktian definitif. Eksploitasi ekosistem *Code-Splitting* teridentifikasi memicu fenomena penolakan kebermanfaatan ganda (*Diminishing Returns*) apabila

diimplementasikan atas platform ringan berkonten statis. Fakta mencatat indeks TBT sistem murni purwarupa tersebut aslinya sudah bertengger damai pada poros 143 ms, menjadikan manuver pemecahan fragmen *JavaScript* tidak menderivasi kelincahan aplikasi, melainkan hanya sekadar menggelembungkan komisi perulangan *Network Requests*. Resolusi tesis ini menyimpulkan bahwa pemisahan asinkron hibrida adalah instrumen pedang bermata satu: sangat adidaya meruntuhkan sumbatan kala digunakan membedah Web berskala dependensi rumit (Modul Besar), namun malah merugikan manakala ditikamkan pada struktur Web Linier biasa.

4.2 Saran Lanjutan dan Pengembangan

Menyadari keterbatasan cakupan spektrum rekayasa teknis pada lapisan ekosistem klien (`Client-Side`), berikut penulis merumuskan usulan perbaikan komprehensif bagi penelitian berantai (*Chain Research*) di institusi untuk agenda di masa serjana/megister berikutnya:

1. **Integrasi Eksekusi Bawah Tanah Bersama Ekosistem *Service Workers* (PWA):** Pada studi mendatang, disarankan agar pemanggilan pra-tunda fungsional (*lazy prefetch*) tidak semata dititipkan pada antrean pasif detektor lenggang peramban (`requestIdleCallback`). Pengiriman muatan dapat diotomasi seutuhnya menuju proksi ketersediaan tembolok luring (*Cache Storage API*) selaras arsitektur hibrida *Progressive Web Application* (PWA). Persilangan absolut kedua pilar ini menjanjikan ilusi waktu unduhan pecahan rute asinkron nol milidetik mutlak (*Zero-Latency Instant Transfer*), mematikan *round-trip delay* untuk halaman antarmuka kunjungan paruh waktu.

2. **Kompensasi Kelambatan dengan Penahan Prioritas Reflow DOM AnimasiVisual:** Pemanfaatan sinkronisasi *Idle Observer* dilaporkan cacat latensi apabila objek pengujian merender sirkuit komputasi antarmuka pergerakan grafis yang konstan (*Continuous CSS Animation / WebGL Canvas*). Kerangka arsitektur masa seberang wajib dibekali pendeteksi mutasi visual khusus (Semacam *MutationObserver Tracker*) demi memadamkan insting *Prefetching* manakala transisi batas ambang bingkai (*Frames Per Second*) melorot di bawah 60 FPS. Ini ditujukan untuk menyeret *Browser* agar menghindari benturan interupsi *micro-stuttering* tatkala transisi kanvas berlangsung.

3. **Komparasi Silang Bersama Kerangka Moderen SSR & Server Edge:** Tesis ini membidik murni optimasi piranti pemrosesan sisi klien (*Client-Side Rendering*). Peneliti pascasarjana di ranah selanjutnya ditantang kuat merumuskan desain kajian persilangan metrik reduksi TBT antara taktik *Vite Bundler CSR* jika didebatkan menggunakan mesin hibrida Server Kompilasi Aktual (semacam pendayagunaan kerangka *Nuxt.js* atau pun *Next.js Server-Side Components*). Peleburan ini dipercaya sanggup menuntaskan anomali pengorbanan waktu awal FCP (Karena HTML dikirim utuh/matang dari Peladen), seraya mengunci kesempurnaan TBT di aras terendah (Karena *Hydration JS Chunk* disuntikkan secara terpartisi asinkron perlahan-lahan ke ruang tunggu perangkat klien).

DAFTAR PUSTAKA

Amenta, V., & Castellani, A. (2019). "Analyzing Total Blocking Time in Modern Web Applications and Its Impact on User Engagement." *Digital Experiences and Software Engineering Journal*, 4(2), 112-126. https://doi.org/10.1109/DESE.2019.2905051

Ardianto, K., & Wibowo, S. (2020). "Integrasi Single Page Application Pada Portal E-Government Skala Daerah Menggunakan Vue.js." *Jurnal Sistem Cerdas dan Informatika Berkelanjutan (J-SCI)*, 7(1), 33-45.

Batool, R., Ahmed, T., & Islam, N. (2021). "Performance Evaluation of Frontend Web Technologies: A Case Study on Single Page Applications vs Multi-Page Architectures." *IEEE Access*, 9, 114521-114530. https://doi.org/10.1109/ACCESS.2021.3105052

Bundschuh, P., Krenn, E., & Schramm, T. (2019). "Impact of Code Splitting on Initial Load Time of Single Page Applications: An Empirical Evaluation." *Journal of Web Application Engineering*, 12(3), 45-61.

Choi, J., & Choi, Y. (2020). "Performance Optimization of E-Government Portals using Lazy Loading and Modular JavaScript." *International Journal of Computer Applications*, 178(9), 23-31. https://doi.org/10.5120/ijca2020920042

Fitriani, A., & Hasanuddin, R. (2021). "Evaluasi Kinerja Sistem Informasi Terdistribusi Pada Arsitektur Micro-Frontend." *Jurnal Sistem Informasi Universitas Hasanuddin*, 14(2), 55-63.

Google Chrome Developers. (2023). "Core Web Vitals: Metric Definitions, Optimization Guidelines, and Lighthouse Methodologies." *Google Web Dev Official Documentation*. Diperoleh tanggal 5 Maret 2026, dari https://web.dev/vitals/

Hasanuddin, U. (2021). *Pedoman Penulisan Tesis dan Disertasi Mahasiswa Pascasarjana Fakultas Teknik Universitas Hasanuddin (Cetak Biru Tahun Berjalan)*. Makassar: Program Studi Magister Teknik Informatika, Universitas Hasanuddin.

Hutagalung, B. (2022). "Analisis Komparatif Performa Framework JavaScript React.js dan Vue.js dalam Ekosistem DOM Virtual." *Jurnal Nasional Teknologi Komputer (JNTK)*, 5(4), 101-115.

Kusumawati, R., Susanti, I., & Darmawan, D. (2022). "Manajemen State Global Menggunakan Pinia pada Pengembangan Aplikasi Pengaduan Masyarakat Terpusat." *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 6(3), 445-452.

Li, X., & Wang, Y. (2020). "Architectural Patterns for Efficient JavaScript Rendering: The Role of the V8 Compliance Engine." *Symposium on Advanced Computing and Communication Systems (SACCS)*, 12, 88-102.

Malavolta, I., et al. (2020). "Code Smells in JavaScript Web Applications: A Systematic Literature Review." *Journal of Web Engineering*, 19(4), 519-548.

Muhammed, S., Lee, K., & Kim, H. (2021). "Asynchronous Dynamic Imports and Route-Level Chunking Strategies for Web Performace." *Proceedings of the IEEE International Conference on Web Technologies*, 211-218.

Pradana, A., & Saputra, E. (2023). "Pengaruh Throttling CPU dan Emulasi Jaringan Lambat Terhadap First Contentful Paint di Lingkungan Headless Browser." *Jurnal Ilmu Komputer dan Informatika*, 11(2), 230-244.

Rahmatulloh, A., Gunawan, R., & Pratama, F. (2019). "Performance Comparison of the REST API and GraphQL in Web Applications." *Journal of Physics: Conference Series*, 1402(6), 066031. https://doi.org/10.1088/1742-6596/1402/6/066031

Setiawan, B. (2021). "Peranan Vite Rollup pada Ekosistem Pemrograman Frontend Skala Enterprise." *Jurnal Informatika Terapan (J-IT)*, 8(1), 12-25.

W3C (World Wide Web Consortium). (2022). "Performance Timeline Level 2: Web APIs for Navigational Tracing." *W3C Working Draft*. Diperoleh dari https://www.w3.org/TR/performance-timeline-2/

Zheng, W., & Li, Y. (2022). "Advanced Code Splitting and Prefetching Lazy Loading Techniques in Modern Frontend Ecosystems." *International Journal of Advanced Computer Science and Applications (IJACSA)*, 13(5), 112-118.